STA 3402 – Machine Learning & Neural Networks

Day 6 Lecture Notes

Recommender Systems (Part 2)

Collaborative Filtering • Jaccard & Cosine Similarity • Normalizing Ratings • UV-Decomposition & RMSE • The Netflix Challenge • Building a Recommender in R

What this lecture covers

1. A quick recap of last lecture (what a recommender system is, content-based vs collaborative filtering, the utility matrix, the computers example).

2. **Collaborative filtering** – how to measure how similar two users are: **Jaccard distance**, **cosine distance**, **rounding the data** and **normalizing ratings**.

3. Class activities in R: normalizing a utility matrix, making a binary (0/1) matrix, and writing a Jaccard distance function.

4. **Dimensionality reduction** – **UV-decomposition** and the **root-mean-square error (RMSE)**.

5. The **Netflix Challenge** (the $1,000,000 prize).

6. Building a real recommender system **from scratch in R** with the MovieLens 100K data set.

7. Homework, exam tips and announcements.

Exam tip

The lecturer said clearly: **Jaccard similarity/distance, cosine similarity/distance, rounding and normalizing** will come in the **final exam** – both **by hand** and **in R**.

The **UV-decomposition / RMSE step-by-step optimisation** part will **NOT be tested** – it is only for your knowledge.

In the lab-based exam you may be given a data set and asked to **build a recommender system**.

Sources used for these notes: the Day 6 lecture recording, the class R code, the whiteboard pages (Day 1 and Day 2), and the textbook chapter *"Recommendation Systems"* (Chapter 9, Mining of Massive Datasets). All calculations below were checked in R.

1. Quick recap of last lecture

The lecture began with a short review of what we learned last time, so that we can continue from there.

1.1 What is a recommender system?

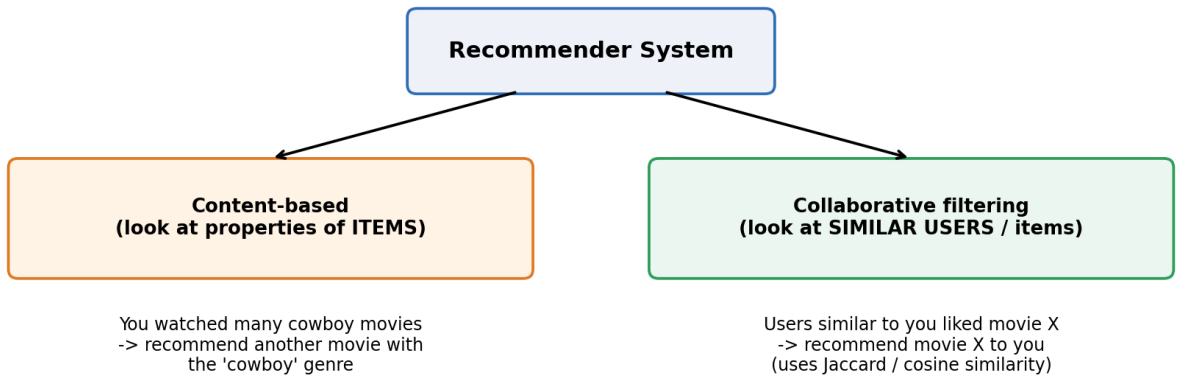
A **recommender (recommendation) system** is a system that **recommends something** to a user – it predicts what a user will like, and suggests those things. Answers given in class (Day 1 whiteboard):

- **Spotify** suggests songs or playlists for you.
- If you **buy a camera** online, the system also suggests **camera accessories**.
- Even choosing a statistical method is a kind of recommendation: e.g. when data contain **outliers**, the **median** is recommended instead of the mean.
- Other textbook examples: offering **news articles** to online newspaper readers based on their interests; offering customers of an online shop suggestions based on their past purchases or searches.

The second Day 1 question was: **What is the Netflix Challenge?** – a competition to improve Netflix's own recommendation system. We will cover it in Section 6.

1.2 Two groups of recommender systems

Recommendation systems use many different technologies, but we can **classify them into two broad groups**. From now on, **whenever you see a recommender system, your first job is to decide whether it is content-based or collaborative filtering** – that is how you learn this topic.



- **Content-based systems** examine the **properties (content) of the items**. Example: if a Netflix user has watched many cowboy movies, recommend a movie that is classified in the database as having the "cowboy" genre – based on the content he watched before.
- **Collaborative filtering systems** recommend items based on **similarity measures between users and/or items**. The items recommended to a user are those preferred by similar users. (Last time we learned about cosine similarity and other measures for this.)

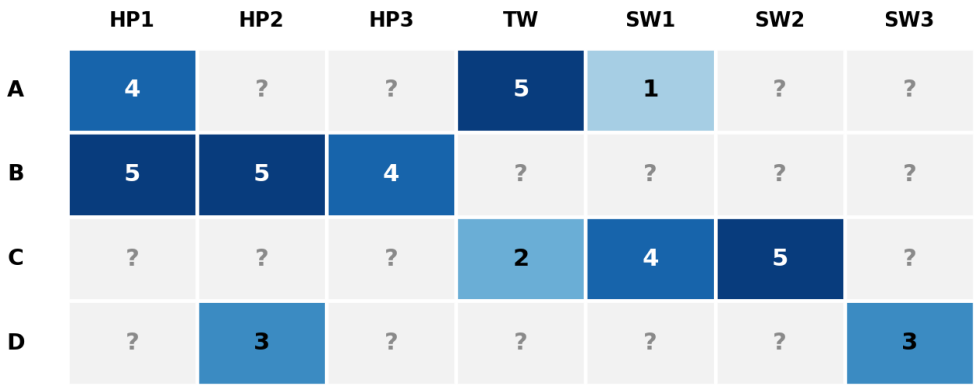
1.3 A model for recommender systems: the utility matrix

Somehow we need to put the data in a meaningful way. The standard way is the **utility matrix**. In a recommender system there are two kinds of things: **users** and **items**. The utility matrix gives, for each user–item pair, a value that shows how much that user likes that item (for example a rating from 1 to 5 stars).

Example (Figure 9.1 / 9.4): 4 users (A, B, C, D) and 7 movies – Harry Potter 1, 2, 3 (HP1, HP2, HP3), Twilight (TW), Star Wars 1, 2, 3 (SW1, SW2, SW3). So it is a **4 × 7 matrix** (4 rows, 7 columns). Ratings are on a 1–5 scale (5 = highest). A **blank** means the user has **not rated** that movie.

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

Utility matrix: users (rows) x movies (columns). '?' = blank (not rated)



- In class the lecturer asked students to rate a recently watched movie (e.g. Spider-Man) from 1 to 5. It depends on your taste: if you like romantic movies you may give 3 or 4; if you like sci-fi you may give 4 or 5.
- Most user–movie pairs are **blank**. In practice the matrix is even more **sparse** – “sparse” means it has **a lot of blanks (or zeros)**, because each user rates only a tiny fraction of all movies.
- **The goal of a recommender system is to fill (predict) the blanks** in the utility matrix. Example: would user A like SW2? A watched SW1 and did **not** like it (rating 1), and liked HP1 and TW. So would A like Star Wars 2? This is the type of question a recommender system answers.
- In many applications we don’t need to predict every blank – we only need to find a few items in each row that are likely to get a **high** rating, and recommend those.

Last time we also discussed the **long tail** (online shops can offer millions of items, while physical “brick-and-mortar” shops can only show the most popular items, so online shops **must** recommend) and **applications** such as product recommendations, movie recommendations and news articles.

1.4 Content-based recap: items as vectors

Content-based recommendation focuses on **properties of items**; collaborative filtering focuses on **relationships between users and items** (i.e. similarity).

Example 9.2 (from last lecture). Suppose the only features of movies are the **set of actors** and the **average rating**. Two movies have five actors each; two of the actors are in both movies. One movie has an average rating of 3 and the other 4. The vectors look like:

	act 1	act 2	act 3	act 4	act 5	act 6	act 7	act 8	avg rating
Movie 1	0	1	1	0	1	1	0	1	3α
Movie 2	1	1	0	1	0	1	1	0	4α

There are, in principle, infinitely many other actors (components with 0 in both vectors), but cosine distance is **not affected** by components where both vectors are 0. α is a **scaling factor** for the rating. The cosine of the angle is:

$$\cos\theta = (2 + 12\alpha^2) / \sqrt{(25 + 125\alpha^2 + 144\alpha^4)}$$

With α = 1 → 0.816; α = 2 → 0.940 (looks more similar); α = 1/2 → 0.619 (looks quite different). So the **choice of scaling factor changes how similar the items look**.

1.5 The computers example – why we need scaling

Exercise 9.2.1. Three computers A, B and C have these numerical features:

Feature	A	B	C
Processor speed	3.06	2.68	2.92
Disk size	500	320	640
Main-memory size	6	4	6

We may imagine each computer as a vector, e.g. A = [3.06, 500, 6]. We can compute the cosine distance between any two vectors, but **the features are not on the same scale**: disk size is in the hundreds while processor speed is around 3. If we do not scale them, **disk size will dominate** the calculation and hide the differences in the other features. So we use a scale factor 1 for processor speed, α for disk size and β for main memory, and compute the cosines in terms of α and β (and then, e.g., for α = β = 1).

> **Note**
>
> If we use α = β = 1 (no scaling), all three cosines are about 0.99999 (angles less than 0.3°), i.e. all computers look almost identical – only because disk size dominates. That is exactly why scaling is needed.

Homework from last lecture (Day 1 whiteboard)

Exercise 9.2.2. Another way of scaling is to **normalize** the vectors: compute the average of each component (feature) and subtract it from that component's value in each vector. (a) Normalize the vectors for the three computers.

Exercise 9.2.3. A user rated the computers: A = 4 stars, B = 2 stars, C = 5 stars. (a) Normalize the ratings for this user. (b) Compute a user profile with components for processor speed, disk size and main memory size.

The lecturer asked the class to send the normalized values and the part (b) answer, saying: *“These things will appear in the final exam as well, so you should know how to do this.”*

Worked answers (added to help you check your work)

9.2.2 (a) Averages: speed = (3.06 + 2.68 + 2.92)/3 = 2.887; disk = (500 + 320 + 640)/3 = 486.67; memory = (6 + 4 + 6)/3 = 5.333.

Normalized: A = [3.06 − 2.887, 500 − 486.67, 6 − 5.333] = **[0.173, 13.33, 0.667]**; B = **[−0.207, −166.67, −1.333]**; C = **[0.033, 153.33, 0.667]**.

9.2.3 (a) The user’s average rating = (4 + 2 + 5)/3 = 11/3. Normalized ratings: A = 4 − 11/3 = **1/3**, B = 2 − 11/3 = **−5/3**, C = 5 − 11/3 = **4/3**.

9.2.3 (b) Weight each computer’s features by the normalized rating and take the average:

speed = (1/3·3.06 − 5/3·2.68 + 4/3·2.92)/3 = **0.149**; disk = (1/3·500 − 5/3·320 + 4/3·640)/3 = **162.2**; memory = (1/3·6 − 5/3·4 + 4/3·6)/3 = **1.111**. So the user profile ≈ [0.149, 162.2, 1.111] – this user likes large disks and more memory.

2. Collaborative filtering

The previous part was **content-based** filtering. Now we take a **significantly different approach**.

- Instead of using **features of items** to decide similarity, we focus on the **similarity of the user ratings** for two items.
- In place of the item-profile vector for an item, we use **its column** in the utility matrix.
- Instead of creating a profile vector for users, we represent each user by **their row** in the utility matrix.
- Users are **similar** if their vectors (rows) are **close** according to some distance measure such as **Jaccard distance** or **cosine distance**.
- A recommendation for a user U is made by looking at the users that are **most similar to U** and recommending items that **these users like**.

Key idea

Collaborative filtering = the process of **identifying similar users** and **recommending what similar users like**.

2.1 Measuring similarity

The first question is: **how do we measure the similarity of users (rows) or items (columns)** in the utility matrix?

If we had two **continuous** variables X and Y (e.g. height and weight), we could measure similarity using the **Pearson correlation**. But here we have ratings with many blanks, so we look at other measures.

We use Figure 9.4 again (same as Figure 9.1). The data is too small to draw reliable conclusions, but its small size makes the **pitfalls** of each distance measure easy to see.

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

Look specifically at users A and C. They rated **two movies in common**: **TW** and **SW1** (highlighted). But they have almost **diametrically opposite opinions**:

- User A gives Twilight **5** (likes it a lot) and Star Wars 1 only **1**.
- User C gives Twilight **2** and prefers Star Wars 1 (**4**) better than Twilight.

We would expect that a **good distance measure should put A and C rather far apart**. A good metric should capture these dissimilarities.

Real-life example from class: dating apps (e.g. Tinder)

The lecturer explained that these similarity methods are used in practice, for example in dating apps like **Tinder**. The app shows a face and some basic details; you **swipe right** if you like the person and **swipe left** if you don't. You can also set a radius (how many kilometres away).

After some time the algorithm looks at the **similarities** (your age, preferences, swipes) and gives you **recommendations**. In some countries people then meet for a coffee to decide whether to continue. The key point: **the algorithm is looking at similarities between users** – exactly what we are doing here.

2.2 Jaccard distance

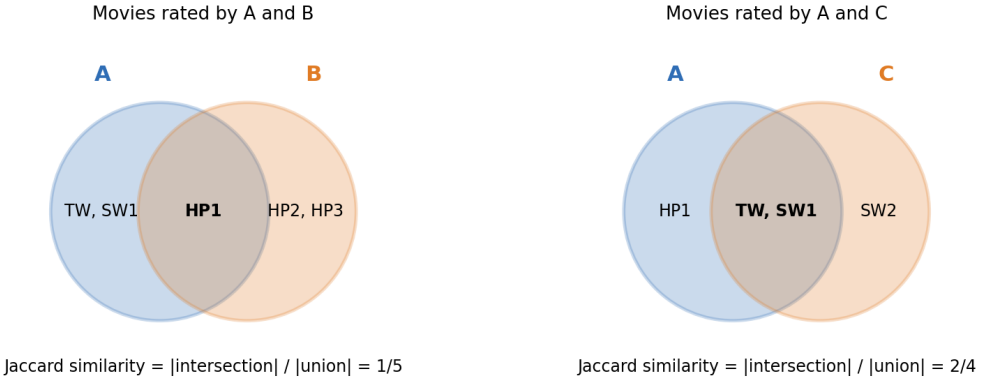
With **Jaccard** we **ignore the values** (the actual ratings) and focus only on the **sets of items rated**.

Jaccard similarity J(A, B) = |A ∩ B| / |A ∪ B| Jaccard distance = 1 − J(A, B)

(∩ = intersection = items in **both** sets; ∪ = union = items in **at least one** set.)

- If the utility matrix only showed **purchases** (bought / not bought), Jaccard would be a good measure to choose.
- But when the utilities are **detailed ratings**, Jaccard distance **loses important information** (it does not care whether the rating was 1 or 5).

Example 9.7 – step by step



1 Movies rated by A = {HP1, TW, SW1}. Movies rated by B = {HP1, HP2, HP3}.

2 Intersection A ∩ B = {HP1} → size **1**. Union A ∪ B = {HP1, HP2, HP3, TW, SW1} → size **5**.

3 Jaccard similarity (A, B) = 1/5 = 0.2, so Jaccard distance = 1 − 1/5 = **4/5** (very far apart).

4 Movies rated by C = {TW, SW1, SW2}. A ∩ C = {TW, SW1} → 2. A ∪ C = {HP1, TW, SW1, SW2} → 4.

5 Jaccard similarity (A, C) = 2/4 = 0.5, so Jaccard distance = 1 − 1/2 = **1/2** (halfway).

So Jaccard says: similarity(A, C) = 0.5 > similarity(A, B) = 0.2, i.e. **A appears closer to C than to B**.

Note

But this conclusion is **intuitively wrong**! A and C **disagree** on the two movies they both watched, while A and B **both liked** the one movie they have in common (HP1: A gave 4, B gave 5). Jaccard fails here because it ignores the rating values.

Exam tip

You should be able to calculate Jaccard similarity and Jaccard distance by hand for any pair of users.

2.3 Cosine distance (cosine similarity)

With cosine we **do** take the rating values into account. We treat **blanks as 0**.

Note

Treating blanks as 0 is **questionable**: it has the effect of treating "not rated" as being **more similar to disliking** the movie than liking it.

$$\cos\theta = (\mathbf{x} \cdot \mathbf{y}) / (\|\mathbf{x}\| \times \|\mathbf{y}\|) = \Sigma\, x_i y_i \,/\, (\sqrt{\Sigma x_i^2} \times \sqrt{\Sigma y_i^2})$$

The top (numerator) is the **dot product** – multiply the ratings of movies that **both** users rated and add them up (blanks are 0, so they contribute nothing). The bottom (denominator) is the length of each vector – square **all** the ratings of each user, add, take the square root.

Example 9.8 – A and B

Common movie: HP1 (A = 4, B = 5). A's ratings: 4, 5, 1. B's ratings: 5, 5, 4.

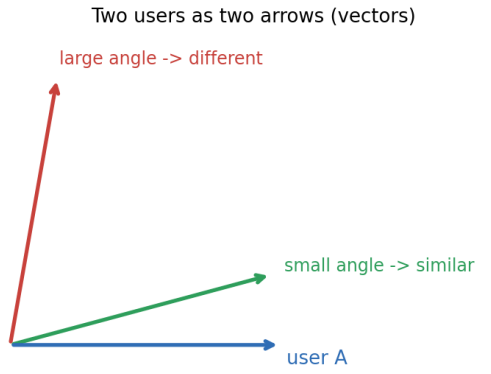
$$\cos(A, B) = (4 \times 5) / (\sqrt{(4^2 + 5^2 + 1^2)} \times \sqrt{(5^2 + 5^2 + 4^2)}) = 20 / (\sqrt{42} \times \sqrt{66}) = 20 / 52.65 = \mathbf{0.380}$$

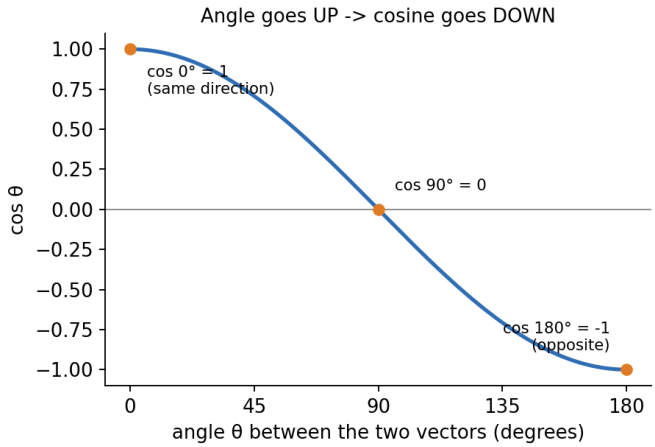
Class exercise – A and C (the lecturer asked the class to do this)

Common movies: TW (A = 5, C = 2) and SW1 (A = 1, C = 4). When there are two common movies, you **add** the products. C's ratings: 2, 4, 5.

$$\cos(A, C) = (5 \times 2 + 1 \times 4) / (\sqrt{(4^2 + 5^2 + 1^2)} \times \sqrt{(2^2 + 4^2 + 5^2)}) = 14 / (\sqrt{42} \times \sqrt{45}) = 14 / 43.47 = \mathbf{0.322}$$

How to interpret the cosine





- Recall cos θ from trigonometry: in a right triangle, cos θ = adjacent / hypotenuse. cos 0° = 1, cos 90° = 0.
- When the **angle increases, the cosine value goes down**.
- So a **larger cosine → smaller angle → smaller distance → the users are closer (more similar)**.
- Here cos(A, B) = 0.380 > cos(A, C) = 0.322, so cosine tells us **A is slightly closer to B than to C** (better than Jaccard, but the difference is small).

Note

In the recording the lecturer once said “small angle means larger distance” – this was a slip of the tongue. The correct rule (as in the textbook) is: **small angle = large cosine = small distance = similar**.

Real-life story from class: a telecom consultancy

The lecturer once did consultancy for a telecom company (like Dialog). They had about 10 million records per month – each unique user (e.g. 07x-xxxxxxx) and the prepaid packages they used (99, 169, …). Some people change packages, some stick to the same one. They needed the **cosine similarity** between users.

On their own laptops (no Google Colab / Kaggle), **R took about 6–8 hours**, while **Python did it in about 30 minutes**. Python is very fast for this type of work.

We will do lab sessions on cosine similarity and you will develop recommender systems soon.

2.4 Rounding the data

We can try to remove the false similarity between movies a user rated **highly** and movies rated **low** by **rounding** the ratings. Instead of treating ratings as numbers 1–5, we turn them into 0/1:

- Ratings **3, 4, 5 → 1** (liked).
- Ratings **1, 2 → treated as unrated** (blank).

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	1			1			
B	1	1	1				
C					1	1	
D		1					1

Figure 9.5: ratings 3, 4, 5 replaced by 1; ratings 1 and 2 omitted.

1 A = {HP1, TW}, B = {HP1, HP2, HP3}. A ∩ B = {HP1} → 1; A ∪ B = {HP1, HP2, HP3, TW} → 4. Jaccard similarity = 1/4, so **Jaccard distance (A, B) = 3/4**.

2 A = {HP1, TW}, C = {SW1, SW2}. **No intersection** → 0; union = 4. Jaccard similarity = 0/4 = 0, so **Jaccard distance (A, C) = 1 − 0 = 1** (as far apart as possible).

Now **C appears further from A than B does**, which is **intuitively correct**. Applying cosine distance to Figure 9.5 gives the same conclusion.

2.5 Normalizing ratings

Look at Figure 9.4 (before rounding) and Figure 9.6 (normalized). The lecturer asked: *“Look at the matrices and see what formula was used to get the normalized values.”*

What does “normalize” mean?

In the computers example the features were on very different scales (memory ≈ 500, speed ≈ 3), so comparing them directly is not good – that is why we normalize. In most cases normalizing means **dividing by some factor** so that the values become comparable (e.g. weights $w_1 \times 5 + w_2 \times 3 + w_3 \times 4 = 1$).

But in Figure 9.6, if you add user A’s normalized values: 2/3 + 5/3 − 7/3 = **0**. So here the normalized values **sum to zero**. For that to happen there must be some positive and some

negative values – the **highest ratings become positive** and the **smallest ratings become negative**.

The lecturer asked students to search “normalization in collaborative filtering”. A student’s answer: **“subtract the average of each row from each element”** – and that is correct!

Key idea

Normalizing in collaborative filtering: from each rating, **subtract the average rating of that user** (the row average, using only non-blank entries).

Low ratings → **negative** numbers; high ratings → **positive** numbers.

Then with cosine distance, users with **opposite views** of the movies they both watched get vectors in **almost opposite directions** → as far apart as possible. Users with **similar opinions** get a **small angle** between them → high similarity.

Normalizing helps the algorithm **differentiate users better**.

Step by step (Figure 9.4 → Figure 9.6)

User	Ratings	Average	Normalized values (rating − average)
A	4, 5, 1	(4+5+1)/3 = 10/3	HP1: 4 − 10/3 = (12−10)/3 = **2/3** TW: 5 − 10/3 = (15−10)/3 = **5/3** SW1: 1 − 10/3 = (3−10)/3 = **−7/3**
B	5, 5, 4	14/3	HP1: **1/3**, HP2: **1/3**, HP3: 4 − 14/3 = **−2/3**
C	2, 4, 5	11/3	TW: **−5/3**, SW1: **1/3**, SW2: **4/3**
D	3, 3	3	HP2: **0**, SW3: **0**

After normalizing: green = liked (above own average), red = disliked

	HP1	HP2	HP3	TW	SW1	SW2	SW3	
A	**2/3**			**5/3**	**-7/3**			avg 10/3
B	**1/3**	**1/3**	**-2/3**					avg 14/3
C				**-5/3**	**1/3**	**4/3**		avg 11/3
D		**0**					**0**	avg 3

Figure 9.6: the utility matrix of Fig. 9.1 with all ratings normalized

Interesting effect: D’s ratings have effectively **disappeared**, because 0 is the same as a blank when cosine distance is computed. D gave only 3’s and did not differentiate between movies, so it is quite possible that D’s opinions are not worth taking seriously.

Example 9.9 – cosine after normalizing

$$\cos(A, B) = (2/3 \times 1/3) / (\sqrt{((2/3)^2 + (5/3)^2 + (-7/3)^2)} \times \sqrt{((1/3)^2 + (1/3)^2 + (-2/3)^2)}) = \mathbf{0.092}$$

$$\cos(A, C) = (5/3 \times -5/3 + (-7/3) \times 1/3) / (\sqrt{((2/3)^2 + (5/3)^2 + (-7/3)^2)} \times \sqrt{((-5/3)^2 + (1/3)^2 + (4/3)^2)}) = \mathbf{-0.559}$$

Under this measure **A and C are much further apart than A and B** (C even has a negative cosine, i.e. an angle of about 124°), and **neither pair is very close**. Both observations make intuitive sense: A and C disagree on the two movies they rated in common, while A and B gave similar scores to their one common movie.

Exam tip

These are essential steps. In real recommender systems we don't do them by hand, but **you should know the theory** – one or two problems like this can be asked to do **manually**, and in the lab exam you may have to do it in R.

2.6 Class activity: normalize Figure 9.4 using R

The lecturer asked: **"Use R to normalize the matrix given in Figure 9.4"** (and send the R code to mark attendance). Here is one way to do it:

R code

```
# Figure 9.4 as a matrix (blank = NA)
M <- matrix(c(4, NA, NA, 5, 1, NA, NA,
              5, 5, 4, NA, NA, NA, NA,
              NA, NA, NA, 2, 4, 5, NA,
              NA, 3, NA, NA, NA, NA, 3), nrow = 4, byrow = TRUE)
rownames(M) <- LETTERS[1:4]
colnames(M) <- c("HP1","HP2","HP3","TW","SW1","SW2","SW3")

userMeans <- rowMeans(M, na.rm = TRUE)   # average of each user (ignore blanks)
userMeans
normM <- M - userMeans                   # subtract each user's average from their row
round(normM, 3)
```

Output

```
       A        B        C        D
3.333333 4.666667 3.666667 3.000000
    HP1   HP2    HP3     TW    SW1   SW2 SW3
A 0.667    NA     NA  1.667 -2.333    NA  NA
B 0.333 0.333 -0.667     NA     NA    NA  NA
C    NA    NA     NA -1.667  0.333 1.333  NA
D    NA 0.000     NA     NA     NA    NA   0
```

0.667 = 2/3, 1.667 = 5/3, −2.333 = −7/3 … exactly Figure 9.6. (`M - userMeans` works because R recycles the vector of 4 means down the columns, so row i gets mean i subtracted.)

2.7 Homework: cosine distance between A and B after normalizing

HW (from the whiteboard)

After normalizing, find the **cosine distance between A and B**: (I) **manually**, (II) **using R**. The lecturer said: "These are exam questions for sure."

(I) Manually: Numerator: only HP1 is common → (2/3)(1/3) = 2/9 = 0.222. ‖A‖ = √(4/9 + 25/9 + 49/9) = √(78/9) = 2.944. ‖B‖ = √(1/9 + 1/9 + 4/9) = √(6/9) = 0.816. cos = 0.222 / (2.944 × 0.816) = **0.092**. Cosine distance = 1 − 0.092 = **0.908** (or, as an angle, arccos 0.092 ≈ 84.7°).

(II) Using R (continuing from the code above):

R code

```
normZero <- normM
normZero[is.na(normZero)] <- 0          # blanks count as 0 for cosine
cosSim <- function(x, y) sum(x * y) / (sqrt(sum(x^2)) * sqrt(sum(y^2)))

cosSim(normZero["A", ], normZero["B", ])             # cosine similarity
1 - cosSim(normZero["A", ], normZero["B", ])         # cosine distance
acos(cosSim(normZero["A", ], normZero["B", ])) * 180 / pi   # angle in degrees
```

Output

```
[1] 0.09245003
[1] 0.90755
[1] 84.69543
```

2.8 Homework 2 (Day 1 whiteboard): Jaccard and cosine for all pairs

Using Figure 9.4 (original ratings, blanks = 0 for cosine). Check your answers:

Pair	Common movies	Jaccard similarity	Jaccard distance	Cosine similarity
A, B	HP1	1/5 = 0.2	0.8	0.380
A, C	TW, SW1	2/4 = 0.5	0.5	0.322
A, D	none	0	1	0
B, C	none	0	1	0
B, D	HP2	1/4 = 0.25	0.75	15/(√66·√18) = 0.435
C, D	none	0	1	0

3. Exercise 9.3.1 and the R class session

Figure 9.8 is a utility matrix: ratings on a 1–5 star scale of eight items (a to h) by three users (A, B, C). This is a **very exam-friendly exercise** – it can be asked to do manually, or to write a function in R.

	a	b	c	d	e	f	g	h
A	4	5		5	1		3	2
B		3	4	3	1	2	1	
C	2		1	3		4	5	3

Compute the following from this matrix:

a Treating the utility matrix as **boolean** (rated / not rated), compute the **Jaccard distance** between each pair of users.

b Repeat part (a), but use the **cosine distance**.

c Treat ratings of **3, 4, 5 as 1** and **1, 2 and blank as 0**. Compute the **Jaccard distance** between each pair of users.

d Repeat part (c), but use the **cosine distance**.

e **Normalize** the matrix by subtracting from each non-blank entry the average value for its user.

f Using the normalized matrix from part (e), compute the **cosine distance** between each pair of users.

> **Link to statistics (the lecturer's side remark)**
>
> Here three users rate the same items – like related (paired) data in statistics. Recall: for 2 independent treatments we use a **t-test**; for more than 2 we use **ANOVA** (mean comparison). For related samples we use paired versions, and when the **parametric assumptions are not met** we go for **non-parametric tests** that compare distributions. Think about these aspects too, because you are learning statistics as a whole.

3.1 In class: part (c) using R – step by step

The lecturer asked students to do **part (c)** in R and send the code (names were called out to take part). Students said they prefer R for this machine-learning class, so **you should be able to do these little things in R**.

Step 1 – store the data in a matrix

R code

```
ratings = matrix(c(
  4, 5, NA, 5, 1, NA, 3, 2,
  NA, 3, 4, 3, 1, 2, 1, NA,
  2, NA, 1, 3, NA, 4, 5, 3
), nrow = 3, byrow = TRUE)
ratings
```

- `matrix()` is the command; first you give the data.
- **Blanks** are usually treated as `NA` (missing values).
- `nrow = 3` gives the number of rows. Giving `ncol` is optional – once you give the data and the number of rows, R can work out the number of columns.

- by`row = TRUE` means the matrix is **filled row-wise**: R fills the first row, then goes to the second row, and so on.

> **Note**
>
> **Careful:** in the class code the rows for B and C were typed differently from the table (B was typed as 3, 4, 3, 1, 2, 1, NA, NA and C as 2, 1, 3, NA, 4, 5, 3, NA – the blanks were in the wrong places). The correct rows, matching Figure 9.8, are the ones shown above: B = NA, 3, 4, 3, 1, 2, 1, NA and C = 2, NA, 1, 3, NA, 4, 5, 3. Always check the positions of the blanks!

Step 2 – add column names and row names (optional)

R code

```
colnames(ratings)            # no column names yet -> NULL
letters                      # "a" "b" "c" ... "z"  (built-in, lower case)
LETTERS                      # "A" "B" "C" ... "Z"  (built-in, capital)
colnames(ratings) = c(letters[1:8])    # a, b, ..., h
rownames(ratings) = LETTERS[1:3]       # A, B, C
ratings
```

Output

```
   a  b  c d  e  f g  h
A  4  5 NA 5  1 NA 3  2
B NA  3  4 3  1  2 1 NA
C  2 NA  1 3 NA  4 5  3
```

The names are not an extra row – there are still only 3 rows; they are just labels. This makes the matrix look exactly like the question. **It is optional – the lecturer said no marks will be deducted if you don't add names.**

Step 3 – element-wise comparison and ifelse()

R does **element-wise** calculation. If you write `ratings > 1`, R compares **each element** with 1 and returns TRUE or FALSE (a boolean matrix). 4 > 1 is TRUE, 1 > 1 is FALSE, and NA stays NA:

R code

```
ratings > 1
```

Output

```
     a    b     c    d     e    f     g    h
A TRUE TRUE    NA TRUE FALSE   NA  TRUE TRUE
B   NA TRUE  TRUE TRUE FALSE TRUE FALSE   NA
C TRUE   NA FALSE TRUE    NA TRUE  TRUE TRUE
```

So we use `ifelse()`: if the rating is 3 or more, give 1, otherwise 0. (A note on naming: people use different conventions, e.g. `binary_ratings` or `binaryRatings` – the second is **camel notation** (second word starts with a capital letter), which the lecturer usually uses. Stick to any one style.)

R code

```
binaryRatings = ifelse(ratings >= 3, 1, 0)
binaryRatings
```

Output

```
   a  b  c d  e  f g  h
A  1  1 NA 1  0 NA 1  0
B NA  1  1 1  0  0 0 NA
C  0 NA  0 1 NA  1 1  1
```

Step 4 – replace missing values (NA) with 0

The question says blanks must also be 0, but we still have NA. `is.na(binaryRatings)` gives a TRUE/FALSE matrix (TRUE where the value is missing). A first try `is.na(binaryRatings) = 0` did not work – you need to put `is.na(...)` **inside the square brackets**, so that R goes exactly to those locations and assigns 0 (a student found this answer):

R code

```
binaryRatings[is.na(binaryRatings)] = 0
binaryRatings
```

Output

```
  a b c d e f g h
A 1 1 0 1 0 0 1 0
B 0 1 1 1 0 0 0 0
C 0 0 0 1 0 1 1 1
```

Now we have the binary matrix. → A liked {a, b, d, g}; B liked {b, c, d}; C liked {d, f, g, h}.

Step 5 – Jaccard distance for each pair

Jaccard distance = 1 − (intersection / union). We need it for **each pair**. How many pairs? With 3 users: **AB, AC and BC** → 3 distances. So it is best to write a **function** that takes the utility matrix and the two users (a and b):

R code

```
# attempt shown in class (does NOT work correctly yet)
JaccardDist = function(UM, A, B){
  intersect(UM[A,], UM[B,]) / union(UM[A,], UM[B,])
}
intersect(binaryRatings[1,], binaryRatings[2,])
union(binaryRatings[1,], binaryRatings[2,])
```

Output

```
[1] 1 0
[1] 1 0
```

Why it does not work: `intersect()` and `union()` work on the **set of distinct values** in the two vectors. Both rows contain only the values 0 and 1, so the result is just "1 0" – it does not count anything. (The lecturer found the examples of `intersect()` using the R help/example code rather than asking ChatGPT.)

What we actually need is to go **column by column** and **count the positions** where both are 1 (intersection) and where at least one is 1 (union). In the lecturer's example (another version of the data) there were 5 common positions out of 8, giving 5/8 – the point is: **get the number of occurrences**, not the distinct values.

HOMEWORK (due in one week, a link will be opened on the LMS)

Write your **own** R function to compute the Jaccard distance between each pair of users (AB, AC, BC) for part (c). Try it yourself first – at the exam you will **not** be able to use ChatGPT or any AI tool.

One possible solution (check yours after trying):

R code

```
JaccardDist = function(UM, A, B){
  inter <- sum(UM[A, ] == 1 & UM[B, ] == 1)   # both liked
  uni   <- sum(UM[A, ] == 1 | UM[B, ] == 1)   # at least one liked
  1 - inter / uni
}
JaccardDist(binaryRatings, "A", "B")
JaccardDist(binaryRatings, "A", "C")
JaccardDist(binaryRatings, "B", "C")
```

Output

```
[1] 0.6
[1] 0.6666667
[1] 0.8333333
```

Note

About AI tools: the lecturer noticed that some submitted answers were clearly obtained from an AI tool (two students gave exactly the same function; one answer did not even handle the missing values because the screenshot uploaded to ChatGPT did not show the blanks). He was not penalising at this point, but asked students to be honest and to write their own code – otherwise you will only memorise code and won't be able to solve new questions in the exam.

3.2 Answers for all parts of Exercise 9.3.1 (checked in R)

Cosine distance here = 1 − cosine similarity (the cosine similarity is shown in brackets).

Pair	(a) Jaccard, boolean	(b) Cosine, raw ratings	(c) Jaccard, 3-5 → 1	(d) Cosine, 3-5 → 1	(f) Cosine, normalized
A, B	4/8 → 0.5	0.399 (0.601)	2/5 → 0.6	0.423 (0.577)	0.416 (0.584)
A, C	4/8 → 0.5	0.385 (0.615)	2/6 → 0.667	0.500 (0.500)	1.115 (−0.115)
B, C	4/8 → 0.5	0.486 (0.514)	1/6 → 0.833	0.711 (0.289)	1.740 (−0.740)

(e) Normalized matrix (user averages: A = 10/3, B = 7/3, C = 3):

	a	b	c	d	e	f	g	h
A	2/3	5/3		5/3	−7/3		−1/3	−4/3
B		2/3	5/3	2/3	−4/3	−1/3	−4/3	
C	−1		−2	0		1	2	0

In (a) the "liked sets" for (a) are the rated sets: A = {a,b,d,e,g,h}, B = {b,c,d,e,f,g}, C = {a,c,d,f,g,h}; every pair shares 4 items out of a union of 8.

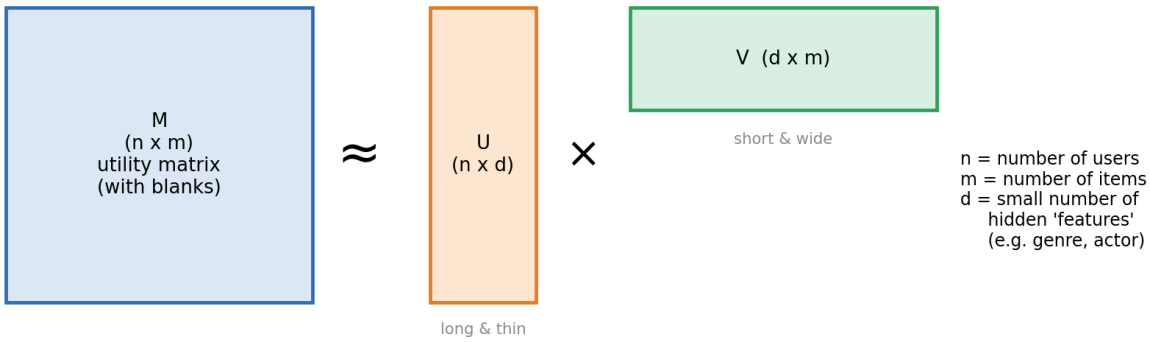
4. Dimensionality reduction: UV-decomposition

We talked about **dimension reduction** some time back in **unsupervised learning** (the graphical ways, and how image quality drops when we reduce the dimensions – the cat picture). Here we use **UV-decomposition**, which is a special case of a more general theory called **SVD (singular-value decomposition)**.

The idea: the utility matrix has **mostly empty** entries. We **decompose** the utility matrix into two matrices U and V that are easier to work with – this is one way to deal with the blanks.

4.1 The idea

- Take movies as an example. **Most users respond to a small number of features**: they like certain **genres**, they may like certain **famous actors or actresses**, and perhaps there are a few **directors** with a significant following.
- Start with the utility matrix **M** with **n rows** (users) and **m columns** (items).
- We might be able to find a matrix **U with n rows and d columns** and a matrix **V with d rows and m columns**, such that **UV closely approximates M** in the entries where M is **non-blank**.
- Then there are **d dimensions** (hidden features) that describe both users and items well. We then use the entries of the product UV to **estimate the blank entries** of M.
- This process is called the **UV-decomposition** of M.



Remember from mathematics: to multiply two matrices, the **number of columns of the first** must equal the **number of rows of the second**. (n × d) × (d × m) gives an (n × m) matrix – the same size as M.

4.2 Example 9.11 – a 5 × 5 matrix

M is a 5 × 5 matrix with **all but two** entries known. We want to decompose it into a **5 × 2** matrix U and a **2 × 5** matrix V (so d = 2 was chosen). The entries of U and V (u_{11}, u_{12}, …, v_{11}, …) are variables to be found.

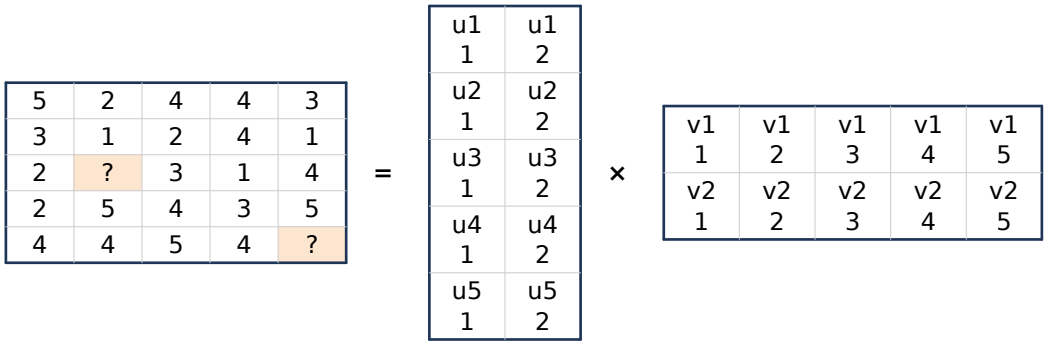


Figure 9.9: UV-decomposition of matrix M (the two "?" cells are blanks).

This is essentially the smallest non-trivial case where there are **more known entries (23) than entries in U and V combined (20)**, so we can expect that the best decomposition will **not** reproduce the non-blank entries of M exactly – it will only **approximate** M.

4.3 Root-mean-square error (RMSE)

We need a measure of how close the product UV is to M. The typical choice is the **RMSE**:

1. **Sum**, over all **non-blank** entries of M, the **square of the difference** between that entry and the corresponding entry of UV.
2. Take the **mean** of these squares by dividing by the number of terms (the number of non-blank entries of M).
3. Take the **square root** of the mean.

Minimising the sum of squares is the same as minimising the RMSE, so often we stop at step 1. **Smaller RMSE is better.**

Example 9.12 – a first (poor) guess: all ones

Suppose we guess that every entry of U and V is 1. Each entry of UV = 1×1 + 1×1 = **2**, so UV is a 5 × 5 matrix of all 2's. This is a **poor guess** – the values of M (5, 4, 3 …) are mostly much larger than 2 – but it makes the RMSE easy to follow.

Row of M	M − UV (differences, blanks ignored)	Squares	Sum of squares
1: 5 2 4 4 3	3, 0, 2, 2, 1	9, 0, 4, 4, 1	18
2: 3 1 2 4 1	1, −1, 0, 2, −1	1, 1, 0, 4, 1	7
3: 2 _ 3 1 4	0, (blank), 1, −1, 2	0, 1, 1, 4	6
4: 2 5 4 3 5	0, 3, 2, 1, 3	0, 9, 4, 1, 9	23
5: 4 4 5 4 _	2, 2, 3, 2, (blank)	4, 4, 9, 4	21
Total			**75**

In row 3 the second column is blank, and in row 5 the last column is blank – those entries are **ignored** when computing the RMSE.

Sum of squared errors = 18 + 7 + 6 + 23 + 21 = **75**. Number of non-blank entries = 25 − 2 = **23**. So RMSE = √(75/23) = √3.26 = **1.806**.

R code (checking the RMSE)

```
M <- matrix(c(5,2,4,4,3, 3,1,2,4,1, 2,NA,3,1,4, 2,5,4,3,5, 4,4,5,4,NA), 5, byrow = TRUE)
UV <- matrix(1, 5, 2) %*% matrix(1, 2, 5)     # all entries = 2
rowSums((M - UV)^2, na.rm = TRUE)            # 18  7  6 23 21
sqrt(mean((M - UV)^2, na.rm = TRUE))         # RMSE = 1.805788
```

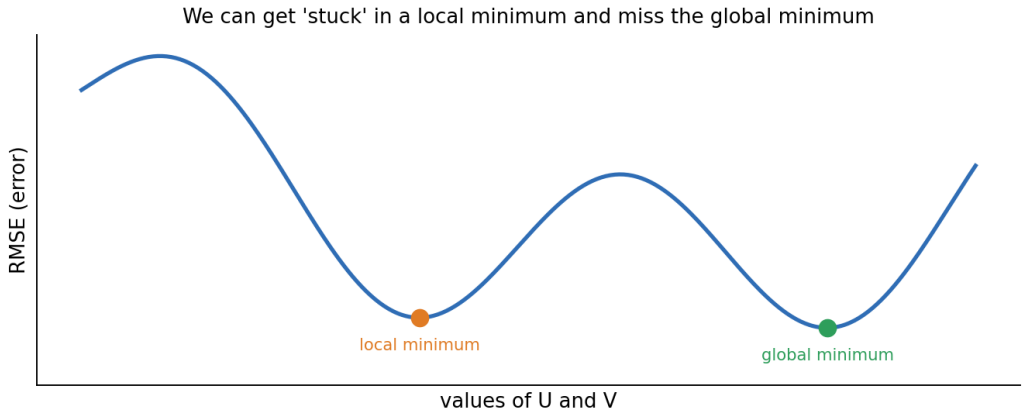
RMSE checks how far UV is **off** from the actual M. We can **iteratively** change U and V, compute the RMSE each time, and choose the best one.

4.4 Finding U and V (for your knowledge – NOT tested)

Exam tip

The lecturer said this RMSE optimisation part is a little complicated and **will not be tested**. The earlier parts (Jaccard, cosine, rounding, normalizing) **will** be tested.

- Start with some **arbitrary** U and V (e.g. all 1's) and **repeatedly adjust** them to make the RMSE smaller.
- We adjust **one element at a time**: make one entry a variable x, write the sum of squared errors as a function of x, differentiate, set to 0 and solve. Example 9.13: making $u_{11} = x$ gives the first-row error $(4 - x)^2 + (1 - x)^2 + (3 - x)^2 + (3 - x)^2 + (2 - x)^2 \to x = \mathbf{2.6}$, which reduces the first-row error from 18 to 5.2 and the total from 75 to 62.2. Then $v_{11} = y$ gives $y = 1.617$, then $u_{31} = z$ gives $z = 1.178$, and so on.
- There will be **many local minima** – values of U and V where no single change reduces the RMSE any more. Only **one** of them is the **global minimum**. We can get stuck at a local minimum without knowing it – a typical problem in optimisation. To improve our chances, start from **many different starting points**.



Building a complete UV-decomposition algorithm involves four decisions:

- **Preprocessing** – normalize M first (subtract user averages and/or item averages); remember to **undo** the normalization when predicting.
- **Initialisation** – e.g. give every element of U and V the value $\sqrt{(a/d)}$, where a is the average non-blank entry of M; add random noise for different starting points.
- **Order of optimisation** – visit elements row-by-row in round-robin, or in a random order; visit them repeatedly.
- **Stopping** – stop when the improvement in RMSE in one round falls below a threshold.

This technique is an example of **gradient descent** (looking at a random fraction of data each time is called **stochastic gradient descent**). To avoid **overfitting** (small RMSE on the given data but poor prediction of future data): move each value only part of the way to its optimum, stop before full convergence, or average the results of several different UV-decompositions. The lecture notes (PDF) also include R routines for this – go through them.

5. The Netflix Challenge

We discussed last time how the Netflix Challenge started. A big boost to research into recommender systems came when **Netflix offered a prize of $1,000,000** to the first person or team to beat their own recommendation algorithm, called **CineMatch**, by **10%**. After over three years of work, the prize was awarded in **September 2009**.

- The data: ratings by about **half a million users** on (small subsets of) about **17,000 movies**, each with a rating (1–5 stars) and the date. Algorithms were tested on a **secret** part of the data.
- **RMSE was used to measure the performance** of the algorithms – that is why we learned RMSE: it is a good metric to **compare** recommender systems.
- CineMatch had an RMSE of about **0.95** – a typical prediction was off by almost one full star. To win, your RMSE had to be **at most 90%** of CineMatch's RMSE.
- **CineMatch was not a very good algorithm.** A very simple algorithm – predict the rating of user u on movie m as the **average of** (1) the average rating given by u on all movies u rated (**person-wise average**), and (2) the average rating of movie m by all users who rated it (**movie-wise average**) – was only **3% worse** than CineMatch.
- The **UV-decomposition** algorithm we just learned was found by **three students** (Michael Harris, Jeffrey Wang and David Kamm) to give a **7% improvement** over CineMatch, when combined with normalization and a few other tricks. (Money encourages a lot of thinking – $1 million was a fortune, especially 15–20 years ago!)
- The **winning entry** was a **combination of several different algorithms** developed independently. A **second team** also had a blend of algorithms that **would have won if it had been submitted a few minutes earlier**. Combining algorithms is a strategy worth remembering.
- Attempts to use **IMDb** (Internet Movie Database) information (actors, directors, 28 genres) were **not useful** – possibly the algorithms discovered that information anyway, and matching movie names between Netflix and IMDb is hard.
- **Time of rating** turned out to be useful – some movies (e.g. "Patch Adams") are liked more by people who rate immediately after watching, others (e.g. "Memento") are appreciated more later.

The lecturer's lesson on deadlines

"This is why I stick to deadlines." In the real world, if you miss a deadline, you miss it. Example: if you plan a shipment at a factory and the container must leave by 5:00 p.m. to reach the port and the ship, and you miss it, you may have to send it by **air freight** – costing far more – and it could cost your job. That second Netflix team lost $1 million by a few minutes. **The lecturer's deadlines are firm.**

The lecturer's advice on learning (and recommender algorithms in your own life)

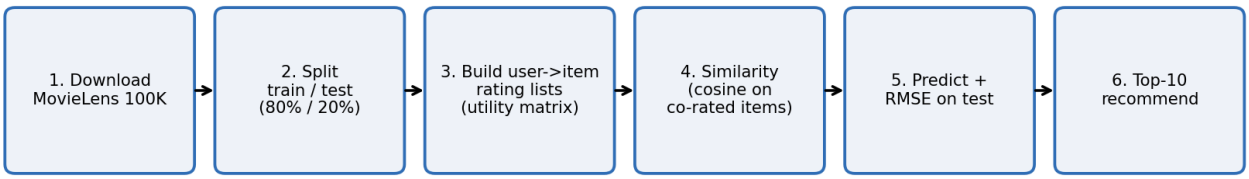
Search YouTube for "Netflix challenge winners" – there are documentaries. If your internet is poor or expensive, download videos in the lab, copy them to your phone and watch while travelling (e.g. on the bus). That is how you learn.

When the lecturer graduated (2008, Faculty of Science), there were very few books and only one or two computers with internet in the library, with long queues. Today you have internet even on your phone – but we are easily **distracted**.

Companies watch us and sell our data and preferences. Their recommender algorithms learn from what we watch: if you watch cooking videos or time-wasting short videos, the algorithm keeps sending you more of the same, and you get stuck. If you follow educational pages and job-posting sites, it will send you those instead. **Tweak your algorithm** – don't waste time; this may be your final year, so bridge the gap before you graduate.

6. Building a recommender system from scratch in R

At the end of the lecture we developed a recommender system from scratch using the **MovieLens 100K** data set (movie ratings). The lecturer used RStudio and shared the code with the class. (He said he was borrowing prepared code, not writing it live – **you don't have to write all of this on your own**, but you should understand each step.)



6.1 Get the data

R code

```
# Create a workspace
dir.create("ml-100k", showWarnings = FALSE)

# Download MovieLens 100K
zip_url <- "https://files.grouplens.org/datasets/movielens/ml-100k.zip"
zip_file <- "ml-100k.zip"
download.file(zip_url, zip_file, mode = "wb")
unzip(zip_file, exdir = ".")

# Load ratings (u.data: user_id, item_id, rating, timestamp)
ratings <- read.table("ml-100k/u.data", sep = "\t",
                      col.names = c("user", "item", "rating", "ts"))
head(ratings)
```

- First check the working directory with `getwd()`; the lecturer set it to the folder of the source file (his "Rajarata D6" folder) so the data would be saved there.
- `download.file()` downloads the zip file and `unzip()` extracts it – both done **from R**. A folder `ml-100k` appears with several data files.
- The ratings are in the file `u.data` (tab-separated). The data description file tells you everything about the data – just follow it.

- Each row has: **user** ID, **item** (movie) ID, **rating** (1–5) and **ts** (timestamp).

6.2 Split the data into train and test

R code

```
set.seed(42)          # so the random split can be reproduced

split_by_user <- function(df, test_frac = 0.2) {
  idx_test <- unlist(tapply(seq_len(nrow(df)), df$user, function(ix) {
    n <- length(ix)
    n_test <- max(1, floor(n * test_frac))
    sample(ix, n_test)
  }))
  test <- df[idx_test, ]
  train <- df[-idx_test, ]
  list(train = train, test = test)
}

spl <- split_by_user(ratings, 0.2)
train <- spl$train
test  <- spl$test

dim(ratings)   # 100000 rows and 4 cols
dim(train)     # 80367     4
dim(test)      # 19633     4
```

- `set.seed(42)` makes the random sampling reproducible.
- For **each user** (`tapply` groups row numbers by user), about 20% of that user's ratings (at least 1) are randomly chosen for the **test** set; the rest go to the **train** set. This way every user appears in both sets.
- The full data has **100,000 rows and 4 columns**; train has about 80% (80,367) and test the remaining 19,633.

6.3 The steps of user-based collaborative filtering

1. Create a **user–item matrix** (the utility matrix).
2. Compute the **similarity between users** (cosine similarity, Jaccard, or Pearson correlation).
3. **Predict** from the neighbours' ratings.

The data is in "long" form (one row per rating: user, item, rating). That is not the form we want – for the utility matrix we need **users on one side, items on the other side, and the rating for each item**.

R code

```
# Build list of item->rating per user
by_user <- split(train, train$user)
user_items <- lapply(by_user, function(d) {
  setNames(d$rating, d$item)
})

head(by_user)
head(user_items)
mode(user_items)     # "list"
```

- `split()` splits the train data into one small data frame **per user** (`by_user`).
- The **apply family** of functions: `apply`, `lapply`, `sapply`, `mapply`, `tapply`. They take a **data source**, (for `apply`) a **direction** (row-wise or column-wise), and a **function** to apply to each part.

- `lapply` returns a **list**. For each user it creates a named vector: the values are the ratings and the names are the item IDs (`setNames(d$rating, d$item)`). This is a memory-friendly way of storing the sparse utility matrix – only the non-blank entries are stored.
- A list cannot be indexed like a matrix; use `user_items[["1"]]` or `user_items$`1`` to get the first member (user 1's ratings).
- Explore data you can't see all at once with summaries, e.g. `summary(ratings$item)`, `max(ratings$item)`. (There are 943 users and 1,682 movies in MovieLens 100K.)

6.4 Baseline model (user bias and item bias)

R code

```
mu <- mean(train$rating)                                    # overall mean rating

user_bias <- tapply(train$rating - mu, train$user, mean)   # how much each user
item_bias <- tapply(train$rating - mu, train$item, mean)   # / item deviates from mu

predict_baseline <- function(u, i) {
  bu <- if (!is.na(user_bias[as.character(u)])) user_bias[as.character(u)] else 0
  bi <- if (!is.na(item_bias[as.character(i)])) item_bias[as.character(i)] else 0
  mu + bu + bi
}

# RMSE on test
pred_base <- mapply(predict_baseline, test$user, test$item)
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))
rmse(test$rating, pred_base)
```

- **mu** = the mean of all training ratings.
- **User bias**: for each user, the average of (rating − mu) – does this user usually rate higher or lower than average?
- **Item bias**: for each item, the average of (rating − mu) – is this movie usually rated higher or lower than average?
- Baseline prediction = **mu + user bias + item bias** (if a user or item is unknown, its bias is 0). This is very similar to the simple "average" algorithm from the Netflix Challenge.
- `mapply` applies the function to each (user, item) pair of the test set. Then we compute the **RMSE** on the test set. In class the RMSE came out at about **0.95**.
- The CF functions below fall back to this baseline, so **define the baseline first** (in class an error "could not find function predict_baseline" appeared because the CF code was run before this part, and `mu` also had to be defined first).

6.5 Cosine similarity and user-based CF



R code

```
# Cosine similarity on co-rated items
cosine_sim <- function(u_vec, v_vec) {
  common <- intersect(names(u_vec), names(v_vec))
  if (length(common) < 5) return(NA_real_)  # require overlap
  u <- u_vec[common]; v <- v_vec[common]
  sum(u*v) / (sqrt(sum(u^2)) * sqrt(sum(v^2)))
}

predict_user_cf <- function(u, i, k = 30) {
  u <- as.character(u); i <- as.character(i)
  if (is.null(user_items[[u]])) return(mu)

  # Find candidate neighbors who rated item i
  neighbors <- names(Filter(function(x) i %in% names(x), user_items))
  neighbors <- setdiff(neighbors, u)
  if (length(neighbors) == 0) return(predict_baseline(u, i))

  sims <- sapply(neighbors, function(v) cosine_sim(user_items[[u]], user_items[[v]]))
  sims <- sims[!is.na(sims)]
  if (length(sims) == 0) return(predict_baseline(u, i))

  top <- head(sort(sims, decreasing = TRUE), k)
  # weighted sum of neighbors' ratings for item i
  r_vi <- sapply(names(top), function(v) user_items[[v]][[i]])
  sum(top * r_vi) / sum(abs(top))
}

pred_ucf <- mapply(predict_user_cf, test$user, test$item)
rmse(test$rating, pred_ucf)
```

- **cosine_sim**: here `intersect()` is used on the **names** (item IDs), so it correctly finds the items **both users rated** (co-rated items). If they share **fewer than 5** items, the similarity is not trusted (NA). Then it applies the cosine formula we learned.
- **predict_user_cf(u, i, k = 30)** – to predict user u's rating for item i:
- 1. If u is unknown, return the overall mean mu.
- 2. Find all other users (neighbours) who **rated item i** (`Filter` keeps only those lists that contain item i; `setdiff` removes u).
- 3. Compute the cosine similarity between u and each neighbour; drop the NA's. If none are left, use the baseline.
- 4. Keep the **top k = 30** most similar neighbours.
- 5. Prediction = **weighted average** of their ratings for item i: Σ(sim × rating) / Σ|sim|.

6.6 Item-based CF

R code

```
by_item <- split(train, train$item)
item_users <- lapply(by_item, function(d) {
  setNames(d$rating, d$user)
})

predict_item_cf <- function(u, i, k = 30) {
  u <- as.character(u); i <- as.character(i)
  if (is.null(item_users[[i]])) return(predict_baseline(u, i))

  # Items that user u has rated
  u_ratings <- user_items[[u]]
  if (is.null(u_ratings)) return(mu)

  rated_items <- names(u_ratings)
  rated_items <- setdiff(rated_items, i)
  if (length(rated_items) == 0) return(predict_baseline(u, i))

  sims <- sapply(rated_items, function(j) cosine_sim(item_users[[i]], item_users[[j]]))
  sims <- sims[!is.na(sims)]
  if (length(sims) == 0) return(predict_baseline(u, i))

  top <- head(sort(sims, decreasing = TRUE), k)
  r_uj <- u_ratings[names(top)]
  sum(top * r_uj) / sum(abs(top))
}

pred_icf <- mapply(predict_item_cf, test$user, test$item)
rmse(test$rating, pred_icf)
```

- This time we split by **item**: for each movie, a named vector of the ratings it got (names = user IDs). These are the **columns** of the utility matrix – remember the **duality**: the same similarity ideas work on rows (users) or columns (items).
- To predict user u's rating for item i: look at the items **u has already rated**, find which of them are **most similar to item i** (cosine similarity between item columns), keep the top 30, and take the weighted average of **u's own ratings** for those items.

6.7 Top-N recommendations for a user

R code

```
# Recommend items the user hasn't rated, using item-CF predictions.
recommend_topN <- function(user_id, N = 10) {
  u <- as.character(user_id)
  seen <- if (!is.null(user_items[[u]])) names(user_items[[u]]) else character()
  all_items <- names(item_users)
  candidates <- setdiff(all_items, seen)

  scores <- sapply(candidates, function(i) predict_item_cf(u, i, k = 30))
  topN <- head(sort(scores, decreasing = TRUE), N)
  data.frame(item = names(topN), score = as.numeric(topN), row.names = NULL)
}

recommend_topN(1, 10)
```

- This is the **main idea** of a recommender: for a user, find the items they have **not seen** (candidates), predict a score for each, and show the **top N** (here 10).
- In class `recommend_topN(1, 10)` first gave an error ("item_users not found") because the item-based part had not been run yet – after running it, it worked and returned the **top 10 items (movie IDs) for user 1 with their predicted scores**.

- It takes a long time to run because there are 100,000 records. **Tip:** put a `print()` inside the function to show intermediate output, so you can see the progress instead of a blank window for hours.

7. Announcements and homework

7.1 Announcements

- **Lab session next Monday** (the tutor has been informed). The batch representative should check with the tutor **in advance** that the lab computers are working. If they are not, inform the lecturer early so no one wastes time going there. If the lab is not available, do it from home / boarding / hostel on your laptop – but going to the lab is recommended.
- **From next week, sit in front of a computer** (bring a laptop or share one). You will be training models – **support vector machines, neural networks** – you can't just watch the code, you must run it. In some programs students are asked to switch on their video to keep focus.
- This course has a significant number of **lab hours**, so hands-on practice is very important.
- **Next lecture:** parametric and non-parametric **supervised learning**, and an introduction to **support vector machines (SVM)**.
- Semester dates: the whiteboard shows **last day of the semester: 20/03/2026**. (In the recording the lecturer also mentioned, as far as he remembered, the last day around the end of February and exams around the second week of March – follow the official faculty notice.)
- Attendance: send your name and registration number (and the requested R code) in the chat.
- **Do the homework now** – if you wait until the last minute you will end up with more than 60 hours of recordings to go through, which is not practical.

7.2 Homework checklist

#	Task	Where in these notes
1	Exercise 9.2.2 (a): normalize the three computer vectors	Section 1.5
2	Exercise 9.2.3 (a), (b): normalize the user's ratings and build the user profile	Section 1.5
3	HW2: Jaccard and cosine similarity for all pairs (Figure 9.4)	Section 2.8
4	Use R to normalize the matrix in Figure 9.4	Section 2.6
5	After normalizing, cosine distance between A and B – (I) manually, (II) using R	Section 2.7
6	Exercise 9.3.1 (a)–(f), especially part (c) in R	Section 3
7	**Write your own Jaccard distance function in R (LMS, due in one week)**	Section 3.1
8	Run the MovieLens recommender code yourself; watch a Netflix-challenge documentary	Sections 5, 6

8. Summary

Concept	In simple words
Utility matrix	Users × items table of ratings; mostly blanks (sparse). Goal: predict the blanks.
Content-based	Recommend items whose **properties** are similar to items the user liked.
Collaborative filtering	Find **similar users** (rows) or **similar items** (columns) and recommend what similar users like.
Jaccard	$\|A \cap B\| / \|A \cup B\|$ on the **sets** of items; ignores rating values. Good for purchase (0/1) data.
Cosine	Dot product / (lengths); uses rating values; blanks = 0. Larger cosine = smaller angle = more similar.
Rounding	3, 4, 5 → 1; 1, 2 → blank/0. Makes Jaccard and cosine give sensible answers.
Normalizing	Subtract each user's average rating: likes → positive, dislikes → negative. Opposite users get opposite vectors.
UV-decomposition	M ($n \times m$) ≈ U ($n \times d$) × V ($d \times m$); d = few hidden features; UV fills the blanks.
RMSE	√(mean of squared errors over non-blank entries). Smaller = better. Used in the Netflix Challenge.
Netflix Challenge	$1M for beating CineMatch (RMSE ≈ 0.95) by 10%; won in 2009 by a blend of algorithms.
R recommender	Split train/test → user/item rating lists → cosine on co-rated items → weighted average of top-k neighbours → RMSE → top-N.

Appendix: the complete class R code (Day 6)

This is the code exactly as shared in class, with only a few comments added. Remember the data-entry correction for the Figure 9.8 matrix (Section 3.1) and the order in which to run the parts (baseline first, then CF, then item-based, then top-N).

Day 6 – class R code

```
# Day 6
#   a b c d e f g h
# A 4 5   5 1   3 2
# B   3 4 3 1 2 1
# C 2   1 3   4 5 3

# store this in a matrix/data frame

ratings = matrix(c(
  4, 5, NA, 5, 1, NA, 3, 2,
  3, 4, 3, 1, 2, 1, NA, NA,
  2, 1, 3, NA, 4, 5, 3, NA
), nrow = 3, byrow = TRUE)
# NOTE: rows B and C above do not match the table (see Section 3.1).
# Correct: NA,3,4,3,1,2,1,NA  and  2,NA,1,3,NA,4,5,3

ratings
colnames(ratings) = c(letters[1:8])

letters
LETTERS

ratings
rownames(ratings) = LETTERS[1:3]
ratings

# Treat ratings of 3, 4, and 5 as 1 and 1, 2, and blank as 0.

binaryRatings = ifelse(ratings>=3,1,0)
binaryRatings

## replace missing with a zero??

binaryRatings[is.na(binaryRatings)] = 0
binaryRatings

# Compute the Jaccard distance between each pair of users.

## JD = 1 - intersection/union
## pairs => AB, AC and BC

# HW: due in a week
JaccardDist = function(UM,A,B){
  intersect(UM[A,], UM[B,])/union(UM[A,], UM[B,])
}

intersect(binaryRatings[1,], binaryRatings[2,])
intersect(binaryRatings[1,], binaryRatings[2,])/union(binaryRatings[A,], binaryRatings[B,])
JaccardDist(binaryRatings,A=1,B=2)

######
# Create a workspace
```

Day 6 – class R code (continued)

```
dir.create("ml-100k", showWarnings = FALSE)

# Download MovieLens 100K
zip_url <- "https://files.grouplens.org/datasets/movielens/ml-100k.zip"
zip_file <- "ml-100k.zip"
download.file(zip_url, zip_file, mode = "wb")
unzip(zip_file, exdir = ".")

# Load ratings (u.data: user_id, item_id, rating, timestamp)
ratings <- read.table("ml-100k/u.data", sep = "\t",
                      col.names = c("user", "item", "rating", "ts"))
head(ratings)

# split the data into train and test

set.seed(42)

split_by_user <- function(df, test_frac = 0.2) {
  idx_test <- unlist(tapply(seq_len(nrow(df)), df$user, function(ix) {
    n <- length(ix)
    n_test <- max(1, floor(n * test_frac))
    sample(ix, n_test)
  }))
  test <- df[idx_test, ]
  train <- df[-idx_test, ]
  list(train = train, test = test)
}

spl <- split_by_user(ratings, 0.2)
train <- spl$train
test  <- spl$test

dim(ratings)
# 100000 rows and 4 cols

dim(train)
#80367     4
dim(test)
#19633     4

# User-based Collaborative Filtering (from scratch)
# Steps
# 1. Create a user-item matrix (sparse-like approach)
# 2. Compute similarity between users (cosine / Pearson)
# 3. Predict from neighbors' ratings

# Build list of item->rating per user
by_user <- split(train, train$user)
user_items <- lapply(by_user, function(d) {
  setNames(d$rating, d$item)
})
```

Day 6 – class R code (continued)

```
head(by_user)
head(user_items)

mode(user_items)

####################
mu <- mean(train$rating)

user_bias <- tapply(train$rating - mu, train$user, mean)
item_bias <- tapply(train$rating - mu, train$item, mean)

predict_baseline <- function(u, i) {
  bu <- if (!is.na(user_bias[as.character(u)])) user_bias[as.character(u)] else 0
  bi <- if (!is.na(item_bias[as.character(i)])) item_bias[as.character(i)] else 0
  mu + bu + bi
}

# RMSE on test
pred_base <- mapply(predict_baseline, test$user, test$item)
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))
rmse(test$rating, pred_base)

# (The rest of the code - cosine_sim, predict_user_cf, by_item / item_users,
#  predict_item_cf and recommend_topN - is exactly as shown in
#  Sections 6.5, 6.6 and 6.7.)
```